

Scalability Roadmap

Production Architecture & Future Phases · Technical Blueprint

Site	Cinerglass Lommel	Date	10 June 2026	Status	Living document	Repo	github.com/Explodingding/cinernet
------	-------------------	------	--------------	--------	-----------------	------	-----------------------------------

Technical Blueprint — Production Architecture & Future Phases

Document	docs/SCALABILITY_ROADMAP.md
Audience	Automation Engineering, OT/IT Integration, Engineering Management
Companion	docs/TPM_ALIGNMENT.md — maintenance-strategy alignment
Status	Living document — phases promoted as they are delivered

1. Single-Line Diagram Standards — Rendering Architecture

The credibility of an electrical engineering tool is decided in the first ten seconds an electrical engineer looks at it. CINERNET's rendering layer is therefore built to the same drawing conventions as EPLAN Electric P8 and AutoCAD Electrical output: **strict orthogonal routing, deterministic parallel-conductor separation, and hierarchy-true vertical placement.**

1.1 Strict 90-degree orthogonal routing

All cable geometry is produced by a custom edge router (`components/topology/PowerCableEdge.tsx`) — Bezier curves are not used anywhere on the canvas. The router emits exclusively horizontal and vertical segments:

Exit condition	Emitted path	SLD analogue
Vertical exit (top/bottom wall)	V → H → V via vertical midpoint	Standard feeder drop between hierarchy rows
Horizontal exit (left/right wall)	H → V → H via horizontal midpoint	Bus-bar tap / horizontal tray run
Co-linear endpoints (ΔX or ΔY < 1 px)	Single straight segment	Direct connection

Wall selection is semantic, not cosmetic. The router inspects the relative positions of source and target nodes (via React Flow's `useInternalNode` internals — live measured geometry, not declared geometry) and chooses exit walls accordingly. Cross-building cables, flagged in the data model as `route.spansBuildings: true`, are *forced* into horizontal exits so that inter-building runs read as tray/bus-bar corridors at the source elevation before dropping to the target — exactly how a riser plan such as SMT-5250 draws them.

Cable identification labels are placed at the midpoint of the **longest segment** of each run, mirroring the EPLAN convention of labelling the dominant corridor rather than a corner.

1.2 Parallel-conductor separation (`parallelIndex` tracking)

Real installations run multiple cables along shared corridors; a renderer that overlays them into a single stroke destroys traceability. CINERNET solves this with a **deterministic two-tier pre-processing pass** (`lib/parallelEdges.ts` — `assignParallelIndices()`), executed before edges reach the render layer:

1. **Exact-pair groups** (`source → target` duplicated): each cable receives `parallelIndex` / `totalParallel` / `parallelBothEnds: true` . The lateral offset is applied at **both** walls, keeping the conductors parallel for their entire run — the power + control pair pattern.
2. **Fan-out groups** (multiple cables leaving one node in the same compass direction): `parallelBothEnds: false` . The offset is applied at the **source wall only**, so cables splay from the panel like conductors leaving a terminal block, each arriving at its own target's natural centre.

The offset itself is symmetric around the wall midpoint:

```
laneOffset = (parallelIndex - (totalParallel - 1) / 2) × 14 px
```

Five feeders leaving a distribution cabinet therefore exit at -28, -14, 0, +14, +28 px — individually traceable by eye, with no random jitter and no frame-to-frame instability. The assignment is **pure and deterministic**: identical topology in, identical geometry out, every render. This is the property that distinguishes engineering drawing software from generic graph visualisation.

1.3 Hierarchy-true placement (layout engine contract)

The layout engine (`lib/siteLayout.ts`) guarantees two invariants that EPLAN users take for granted:

- **Vertical truth**: screen Y is derived from physical elevation (mezzanine +5 m → top, ground 0 m → middle, basement -8 m → bottom) and, within each floor band, from the seven-rank electrical hierarchy (`LAYER_RANK` : mv-feed = 0 ... load = 6). Power flows visually upward from the substation.
- **Branch symmetry**: a two-pass tree algorithm (bottom-up leaf counting, top-down X assignment) centres every parent over the total width of its downstream subtree, splitting children symmetrically about the branch centre-line.

No coordinate on the canvas is hand-authored. See `TPM_ALIGNMENT.md` §6 for why this is treated as a quality gate.

2. SCADA & Industrial IoT Integration Lifecycle

2.1 Foundation already in place: `externalRefs`

Every node and edge in the model carries an optional, typed identity block:

```
// types/topology.ts
export interface ExternalRefs {
  scadaTag?: string;
  osapiensAssetId?: string;
}
```

These are populated throughout the live dataset (e.g. `LOMMEL.TR-DP1-1.STATUS` / `AST-TR-DP1-1` on the F10 PFC panel). This is the **join key architecture**: CINERNET elements are addressable from the SCADA namespace and the osapiens asset-management namespace *today*, before any live connection exists. Integration is a wiring exercise, not a remodelling exercise.

2.2 Integration lifecycle — four phases

Phase A — Static twin (current state)

Statuses are set by humans (fault injection UI, DetailDrawer controls) and distributed to all viewers through the render pipeline. The cascade engine, SLD renderer, and progressive disclosure are fully operational. This phase exists deliberately: it proves the visual and data architecture under demo and training load before a single PLC byte is consumed.

Phase B — Read-only telemetry overlay

Transport blueprint. Two interchangeable northbound paths, selected per site-IT policy:

```
PLC / RTU / protection relays
  | (OPC UA / Modbus TCP – OT network)
  ▼
Edge gateway (e.g. Kepware, Ignition, or a slim Node service)
  |
  ├── Path 1: MQTT broker (Sparkplug B topics)
  |   spBv1.0/lommel/DDATA/utility/TR1-1
  |   payload: { amps_l1, amps_l2, amps_l3,
  |             volts, temp_winding, ts }
  |
  └── Path 2: REST/WebSocket API layer
      GET /api/v1/telemetry?tags=LOMMEL.TR_DP1_1.*
      WS  /api/v1/stream    (server push, ~1 Hz)
  ▼
CINERNET telemetry adapter (browser/WebSocket client or SSE)
  | resolves topic/tag → element via externalRefs.scadaTag
  ▼
TelemetryContext → DeviceNode / PowerCableEdge overlays
```

Data contract. A single normalised reading shape, regardless of transport:

```
interface TelemetryReading {
  scadaTag: string;          // join key → externalRefs.scadaTag
  metric: 'amperage' | 'voltage' | 'temperature' | 'power-factor';
  value: number;
  unit: 'A' | 'V' | '°C' | '-';
  quality: 'good' | 'stale' | 'bad'; // OPC quality propagated, never hidden
  timestamp: string;          // ISO 8601, source-clocked
}
```

Rendering rule. Telemetry is an *overlay*, architecturally identical to today's derived statuses: computed into render props, never written into the topology model. The poka-yoke guarantee (no stored derived state, no stale data presented as fresh — `quality: 'stale'` renders greyed) is preserved through the integration.

Phase C — SPC boundaries on live channels

Each monitored channel gains Statistical Process Control limits, stored per element alongside the existing specs:

```
interface SpcBoundary {
  metric: TelemetryReading['metric'];
  target: number;          // process centre line (CL)
  ucl: number;             // upper control limit (e.g. CL + 3σ)
  lcl: number;             // lower control limit
  uclWarning?: number;     // optional 2σ pre-alarm band
  basis: 'nameplate' | 'commissioning-baseline' | 'rolling-30d';
}
```

The `DetailDrawer` gains a live strip-chart per channel with CL/UCL/LCL bands drawn in — turning every device card into a miniature control chart. Limits derive from three auditable bases: nameplate ratings (hard physics), commissioning

baselines (the plant's own as-built normal), or rolling statistical windows (adaptive, seasonal-aware).

Phase D — Closed-loop work-order generation

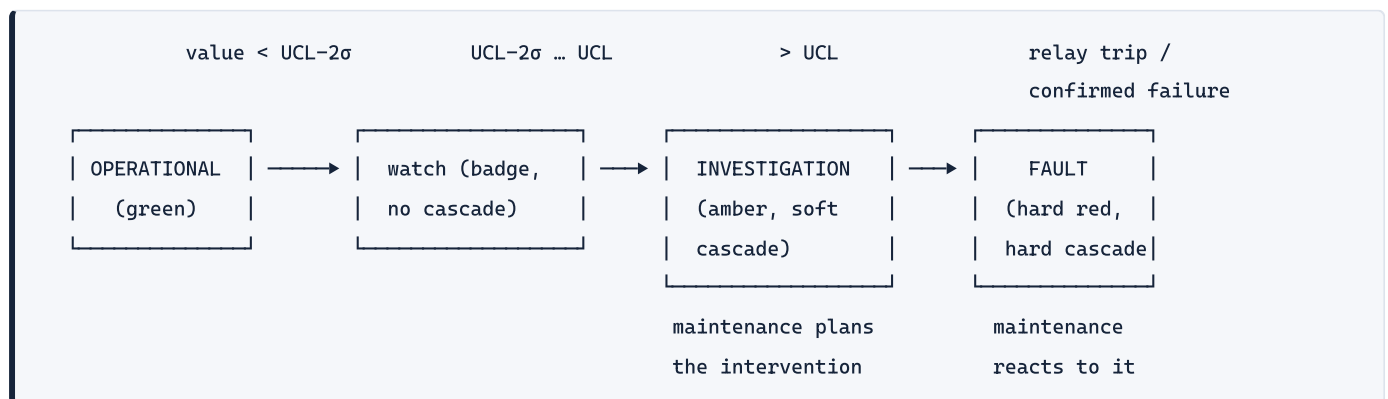
SPC violations and cascade events post outbound to the CMMS / osapiens via `osapiensAssetId`, carrying the affected-equipment list computed by the cascade engine. The work order arrives pre-scoped — the planner sees not just "TR1.1 winding temperature trending high" but the full downstream inventory that a TR1.1 outage would take with it, enabling intelligent scheduling against production windows.

2.3 Why the graph makes integration cheap

A point-to-point SCADA HMI must be *drawn* for every screen. CINERNET's topology is *data*; a new telemetry channel is one `scadaTag` string on one element. The rendering, alarm propagation, parallel-lane routing, and tier filtering are inherited automatically. Integration effort scales with the number of *signals*, not the number of *screens* — the defining economic property of model-driven HMIs.

3. Predictive Maintenance Phase — From SPC Excursion to Planned Intervention

The status vocabulary was designed from day one with three levels — `operational` | `investigation` | `fault` — precisely so that predictive logic would have a **soft tier** to land in. The escalation ladder:



Mechanics of the amber cascade. When a channel crosses its UCL with sustained persistence (debounced — e.g. 3 consecutive readings or 5 minutes, to reject transients), the element's status is raised to `investigation`. The existing BFS engine then propagates `derived-investigation` downstream — the same algorithm, the same rendering pipeline, already implemented and demo-proven. The map shows, in soft amber:

- the deviating asset (e.g. TR1.1, winding temperature trending through UCL),
- every downstream board and load that *would* be lost if it tripped — the would-be blast radius of the *not-yet-failure*.

The operational payoff is scheduling power. Maintenance sees the full consequence map while the equipment is still running, and can:

1. order parts against the specific asset (manufacturer and ratings are on the card),
2. bundle the intervention with other due work in the same physical area (location data per node — see `TPM_ALIGNMENT.md` §3),
3. select the production window in which the downstream loss is cheapest,
4. brief the crew with the cascade map as the isolation/restoration plan.

The difference between the amber path and the red path is the difference between a **planned 2-hour window** and an **unplanned multi-shift outage** with thermal risk to the furnace. Severity precedence is already implemented: if a hard fault occurs elsewhere during an amber episode, `derived-fault` outranks `derived-investigation` on any shared equipment — the merged picture stays correct.

Beyond thresholds. Because Phase B archives telemetry with timestamps, later stages can replace static UCLs with trend-based estimators (rate-of-change alarms, remaining-useful-life regressions on thermal data). These slot into the same escalation ladder — they are alternative *triggers* for the same `investigation` state, requiring zero changes to the visualisation or cascade layers.

4. Multi-Wing Architecture Expansion

The site model (`data/buildings.ts` , `MAP_COLUMN_ORDER` , and the per-column allocation in `lib/siteLayout.ts`) was dimensioned for the full Lommel campus from the outset. Buildings are columns; the layout engine computes each column's width from its actual visible node tree, so wings grow without manual re-drafting.

4.1 Spatial allocation

FURNACE 10	UTILITY	FURNACE 20	BATCH HOUSE	WAREHOUSE
(active)	(centre spine)	(future)	+CULLET TOWER	(reserve)
			(subsystem)	
left wing	MV intake, TR	right wing	peripheral	rear
	fleet, main LV	mirror	offset	abstract

← cross-building tray runs (spansBuildings: true) →

Wing	Status	Architecture notes
Utility	Live, fully modelled	The electrical spine: MV feed (basement –8 m), 35 kV switchgear, transformer fleet TR1.1–TR1.3 + TR-SPARE, main LV panels. Centre column — every other wing attaches to it.
Furnace 10	Live, fully modelled	PFC panels TR-DP1.1–1.3, distribution boards F1-MDP-1...9 plus zone boards (HOT/COLD/GEN/UO/MZ), Tier 3 loads (cooling pumps, CAF fans, batch drives). The reference implementation for every future wing.
Furnace 20	Reserved column, ID registered	Designed as a structural mirror of F10: same <code>TopologyLayer</code> hierarchy, same zone vocabulary (<code>furnace-20-ground</code> , <code>furnace-20-elevated</code> already in <code>LocationZone</code>). Population is a data-entry exercise validated by the type system — no engine changes.
Batch House + Cullet Tower	Grouped subsystem, minimal nodes	Modelled as one peripheral subsystem (material-handling loads: conveyors, crushers, elevators). Offset placement reflects their auxiliary electrical role.
Warehouse	Abstract placeholder	<code>BuildingId</code> and zone registered; column allocated. Activation requires only node/edge data.

4.2 Why expansion is cheap — the contract list

Adding Furnace 20 (the largest future scope) requires exactly:

- Data files** — `data/installations/furnace-20.ts` following the F10 pattern. The compiler enforces completeness (location, specs, troubleshooting steps are required fields).
- Cross-building feeders** — edges from Utility LV panels with `route.spansBuildings: true` ; the orthogonal router and parallel-lane assigner handle the corridor geometry automatically.
- Nothing else.** Layout (tree-centred column sizing), tier filtering, cascade propagation, SLD routing, parallel separation, zone striping, and the `DetailDrawer` all operate on the new data without modification — each is driven by the schema, not by per-building code.

The same contract applies to deepening existing wings (Tier 3 build-out of Batch House) and to the telemetry phases of §2: **CINERNET scales by adding data, not by adding code.** That property — enforced by the closed type system and the derived-everything rendering architecture — is the engineering case for treating this application as plant infrastructure rather than as a demo.

5. Phase Summary

Phase	Scope	Engine changes required
A (now)	Static twin: SLD rendering, cascade, tiers, fault drills, per-asset docs & history	—
B	Read-only telemetry overlay (MQTT/Sparkplug B or REST/WS), <code>externalRefs</code> join	Telemetry adapter + overlay context
C	SPC boundaries, live control-band charts in DetailDrawer	<code>SpcBoundary</code> schema + chart component
D	Predictive amber cascades, CMMS work-order posting	Threshold/debounce service; cascade & UI unchanged
E	Furnace 20 + Batch House Tier 3 + Warehouse activation	Data only